

Tu modelo de Machine Learning en la web

En esta clase entiendes qué hace el [Predictor Saber 11](#), aprendes a leer sus resultados y publicas tu propio modelo, el de tu proyecto final, en una página con dirección propia. Solo necesitas un navegador, una cuenta de GitHub y una de Google.

Tu página quedará en: <https://tu-usuario.github.io/mi-modelo/>

PARTE 1 · 10 MIN

Qué hace el modelo

Árboles y Gradient Boosting

PARTE 2 · 10 MIN

Cómo leer los resultados

Los indicadores de la página

PARTE 3 · 40 MIN

Publica tu modelo

4 pasos, sin escribir código

Qué hace el Predictor Saber 11

Recibe 9 respuestas sobre el hogar y el colegio de un estudiante y estima su **puntaje global del Saber 11**, de 0 a 500. Aprendió de 426.052 resultados reales del ICFES.



Entran 9 respuestas

Estrato, educación de los padres, internet, computador, tipo de colegio, jornada, zona y departamento



Un Gradient Boosting

100 árboles de decisión que trabajan en equipo



Sale un puntaje

Con su margen de error y lo que sumó o restó cada respuesta

¿Qué es un árbol de decisión?

Un diagrama de preguntas de sí o no. Cada respuesta lleva por una rama hasta una **hoja**, que tiene un número: el puntaje promedio de los estudiantes que llegaron ahí. Este es un árbol real de dos preguntas, entrenado con los mismos datos:

Un árbol de dos preguntas entrenado con los 426.052 estudiantes de entrenamiento (promedio general: 250 puntos)

¿Hay computador en el hogar?

No

Sí

¿Hay internet en el hogar?

¿La jornada es completa?

NO

226

puntos en promedio
112.606 estudiantes

sí

243

puntos en promedio
96.184 estudiantes

NO

258

puntos en promedio
172.440 estudiantes

sí

294

puntos en promedio
44.822 estudiantes

Con solo dos preguntas, el error promedio baja de 42,9 a 39,3 puntos. El Predictor usa 100 árboles con muchas más preguntas y baja el error a 34,7.

Árbol, bosque y Gradient Boosting

Árbol de decisión

Bosque aleatorio

Gradient Boosting

Un solo diagrama de preguntas. Fácil de explicar, pero se equivoca más.

Muchos árboles distintos, cada uno con una parte de los datos. La estimación es el promedio de todos.

Árboles pequeños en fila: el primero estima grueso y cada árbol nuevo corrige un poco el error de los anteriores. Es el que usa el Predictor.

¿Por qué funciona sin servidor?

Un árbol son preguntas y números. Los 100 árboles viajan en el archivo `modelo.js` y el navegador los recorre en un instante: la página es gratis, rápida y privada, porque las respuestas no salen del computador de quien la usa. En la [ficha del modelo](#) puedes ver las preguntas que hace el árbol 1 con tus respuestas.

Cómo leer los resultados

Abre el [Predictor](#) y busca estos indicadores. Los números son los del modelo publicado, medidos con 106.514 estudiantes que el modelo nunca vio.

La estimación

El número grande del medidor: lo que el modelo espera, en promedio, para estudiantes con esas respuestas.

Con las respuestas más comunes: 260 puntos.

El rango aproximado

La estimación ± 35 puntos, el error típico del modelo.

El puntaje real cayó dentro del rango en 56 de cada 100 estudiantes.

Qué suma y qué resta

Cuánto subió o bajó la estimación por cada respuesta. El modelo parte del promedio y las barras suman la estimación.

250 de partida + 10 de las respuestas = 260.

MAE (error absoluto medio)

En promedio, ¿por cuánto se equivoca? Menos es mejor.

34,7 puntos.

La línea base

Un «modelo» que siempre responde el promedio. Sirve para saber si el nuestro vale la pena.

Se equivoca 42,9 puntos: el nuestro, un 19 % menos.

R²

Qué parte de las diferencias entre estudiantes explica el modelo (0 = nada, 1 = todo).

0,32: explica el 32 %. El resto depende de lo que el modelo no ve.

Verificación «5 de 5»

La página calcula lo mismo que Python (scikit-learn) con 5 estudiantes de prueba. Si no coincidiera, no se publicaría.

Si predices una clase

Si el modelo estima sí/no o una clase (aprobado, reprobado...), la página muestra una **probabilidad** de 0 a 100 % y la ficha agrega la **exactitud**: de cada 100 casos, cuántos acierta.

¿Qué modelo se equivoca menos?

Error promedio (MAE) en los 106.514 estudiantes de prueba, en puntos. Menos es mejor.

Línea base: siempre el promedio		42,9
Árbol de decisión (uno solo)		36,5
Bosque aleatorio (50 árboles)		35,9
Regresión lineal		35,5
Gradient Boosting · el del Predictor		34,7

Gana el Gradient Boosting, pero por poco. Con 9 preguntas sobre el contexto, la información tiene un techo: ningún modelo adivina lo que no está en los datos, como el esfuerzo o el colegio concreto. Un modelo más complejo expresa un poco más, pero no hace magia.

Parte 3 · 40 minutos

Publica tu modelo en 4 pasos

Con los datos de tu proyecto final y el tipo de modelo que usaste. No escribes código: copias una plantilla, llenas un formulario y subes un archivo.



Repositorio

La carpeta de tu proyecto en GitHub.
Tu página vive en su carpeta `web`.

Commit

Guardar un cambio con una nota corta, como «subo mi modelo».

GitHub Actions

Un asistente que revisa tu modelo y publica la página cada vez que cambias algo.

Colab

Python en el navegador, de Google.
No se instala nada.

❗ **Antes de la clase:** crea tu cuenta en github.com/signup (<https://github.com/signup>) (elige un usuario corto: será parte de tu dirección) y ten a mano tu cuenta de Google.

1

Copia la plantilla y activa tu página

10 minutos · en GitHub

1

Con tu sesión iniciada, abre la plantilla:

github.com/Daniel01010101010101/Ejercicio_ejemplo_machine_learning
(https://github.com/Daniel01010101010101/Ejercicio_ejemplo_machine_learning)

2

Pulsa el botón verde **Use this template** → **Create a new repository**. Nombre: `mi-modelo`. Deja **Public** y pulsa **Create repository**.

3

En tu repositorio: **Settings** → **Pages** → en **Source** elige **GitHub Actions**.

4

Abre **Actions** → **Publicar en GitHub Pages** → **Run workflow** y el botón verde **Run workflow**. Espera la ✓ verde (uno o dos minutos).



Listo si en `https://tu-usuario.github.io/mi-modelo/` ves el Predictor Saber 11 (tu copia).



Si en **Actions** ves una **X** roja antes del punto 3, es normal: la página todavía no estaba activada.

2

Entrena tu modelo en Colab

20 minutos · en Google Colab

- 1 Abre el cuaderno plantilla e inicia sesión con Google:

[Abrir plantilla_tu_modelo.ipynb en Colab](https://colab.research.google.com/github/Daniel01010101010101/Ejercicio_ejemplo_machine_learning/blob/main/notebooks/plantilla_tu_modelo.ipynb)

(https://colab.research.google.com/github/Daniel01010101010101/Ejercicio_ejemplo_machine_learning/blob/main/notebooks/plantilla_tu_modelo.ipynb)

- 2 **La primera vez no cambies nada.** Menú **Entorno de ejecución** → **Ejecutar todas** (en inglés: **Runtime → Run all**). Si aparece un aviso, pulsa **Ejecutar de todos modos**. Usa un ejemplo de precios de diamantes.
- 3 **Con tus datos:** llena el formulario «1. Configura tu modelo». Estos son los campos que importan:

```
TITULO      = "Predictor de ..."      # el nombre de tu página
URL_DATOS   = ""                     # vacío = aparece un botón
para subir tu CSV
OBJETIVO    = "columna_a_predecir"    # lo que quieres predecir
VARIABLES   = "col_1, col_2, col_3"   # de 3 a 10 columnas (vacío
= el cuaderno elige)
MODELO      = "boosting"              # boosting, bosque, arbol o
lineal
```

- 4 **Ejecutar todas** otra vez. Mira la tabla: los cuatro modelos contra la línea base. El cuaderno te dice cuál se equivoca menos.
- 5 Al final se descarga `modelo.js`. Si el navegador pregunta, permite la descarga.

✓ **Listo si** el cuaderno dice «OK: JavaScript y scikit-learn coinciden» y tienes `modelo.js` en tus descargas.

3

Sube tu modelo

5 minutos · en GitHub

- 1 En tu repositorio, pestaña `Code`, abre la carpeta `web`.
- 2 `Add file` → `Upload files` y arrastra tu `modelo.js`.
- 3 Pulsa el botón verde `Commit changes`.
- 4 Abre `Actions` y espera la ✓ verde. Antes de publicar, GitHub revisa que tu modelo calcule lo mismo que Python.

✓ **Listo si** la ejecución más reciente en `Actions` tiene una ✓ verde.

❗ Si el archivo quedó como `modelo (1).js`, cámbiale el nombre a `modelo.js` antes de subirlo. Tiene que quedar dentro de la carpeta `web`.

4

Abre y comparte

5 minutos

- 1 Abre tu página y recarga sin caché: `Ctrl + F5` en Windows, `Cmd + Shift + R` en Mac.
- 2 Revisa que salgan tu título y tus preguntas, y que **Verificación** diga «5 de 5 coinciden».
- 3 Comparte tu enlace con un compañero y ábrelo en tu celular.

✓ **Listo si** otra persona abre tu enlace y ve tu modelo funcionando.

¿Sirve mi proyecto?

- Un archivo **CSV** con una fila por caso y los nombres de las columnas en la primera fila. Desde Excel: `Archivo → Guardar como → CSV UTF-8`.
- La columna que predices puede ser un **número** (un precio, un puntaje, unas horas): la página estima ese número. O **sí/no** o una **clase**: la página estima la probabilidad. Si hay más de dos clases, escribe en `CLASE_POSITIVA` la que te interesa.
- De 3 a 10 columnas que la expliquen, de texto (una ciudad, una categoría) o de números (una edad, un tamaño).
- No uses datos personales (nombres, documentos, correos) ni columnas como género o etnia para predecir: tu página es pública.

¿Y si ya entrené mi modelo en otro cuaderno?

Pasa tus datos limpios a un CSV y elige en la plantilla el mismo tipo de modelo. El cuaderno lo entrena igual y lo convierte en `modelo.js`. La página necesita el modelo como números, no un archivo `.pkl`.

Si algo falla

No veo el botón «Use this template»

Revisa que iniciaste sesión en GitHub. Si aun así no aparece, usa **Fork** (arriba a la derecha). En una copia hecha con Fork, abre **Actions** y pulsa **I understand my workflows, go ahead and enable them** antes de activar la página.

Mi página muestra «404»

Espera uno o dos minutos y recarga. Si sigue: revisa que en **Settings → Pages** la fuente sea **GitHub Actions**, que en **Actions** haya una ✓ verde y que la dirección tenga tu usuario en minúsculas, el nombre exacto del repositorio y la barra final.

En «Actions» hay una X roja

Ábrela y mira qué paso falló. **«Configurar GitHub Pages»:** falta activar la página (paso 1, punto 3) y volver a pulsar **Run workflow**. **«Prueba de paridad»:** el `modelo.js` no es válido; descárgalo otra vez desde Colab y súbelo sin abrirlo ni editarlo.

Subí mi modelo pero la página sigue igual

Recarga sin caché (**Ctrl + F5**). Revisa que el archivo se llame exactamente `modelo.js`, que esté dentro de la carpeta `web` y que la última ejecución en **Actions** esté en verde.

Colab dice «No encuentro la columna» o «Tu objetivo tiene N clases»

Copia el nombre exacto de la columna desde la tabla del cuaderno (mayúsculas, tildes y espacios incluidos). Si tu objetivo tiene más de dos clases, escribe en `CLASE_POSITIVA` la clase cuya probabilidad quieres estimar.

Mi modelo casi no le gana a la línea base

Tus columnas explican poco lo que quieres predecir. Prueba otras columnas en `VARIABLES`, o déjalo vacío para que el cuaderno elija las que más explican.

No logro correr Colab en clase

Sigue con los pasos 3 y 4 usando el modelo de ejemplo ya listo: en tu repositorio abre `ejemplos/diamantes/modelo.js`, descárgalo (botón de descarga arriba a la derecha del archivo) y súbelo a la carpeta `web`.

Lista final

Marca cada punto cuando lo termines. Se guarda en este navegador.

☐ Entiendo qué hace un árbol de decisión y un Gradient Boosting.

☐ Sé leer la estimación, el MAE, la línea base y el R^2 .

☐ Copié la plantilla y veo el Predictor en mi dirección.

☐ Entrené mi modelo en Colab y descargué `modelo.js`.

☐ Subí `modelo.js` a la carpeta `web` y Actions quedó en verde.

☐ Mi página dice «5 de 5 coinciden» y compartí mi enlace.

0 de 6 listos.

Uso responsable

Tu página es pública y cualquiera puede usarla. Escribe en ella qué hace el modelo y qué no: sus estimaciones tienen error, describen asociaciones (no causas) y no deben usarse para tomar decisiones sobre personas.